TEAM 3 - PROJECT IDEA

TEAM 3 IDEA - REAL ESTATE

A real estate matchmaking platform that learns your exact preferences through a swipe-based interface. It solves a major gap in the market by going beyond standard housing metrics, focusing heavily on the interior reality of the home—including architectural blueprints, detailed interior aesthetics, and the actual condition of the space.

Product Features

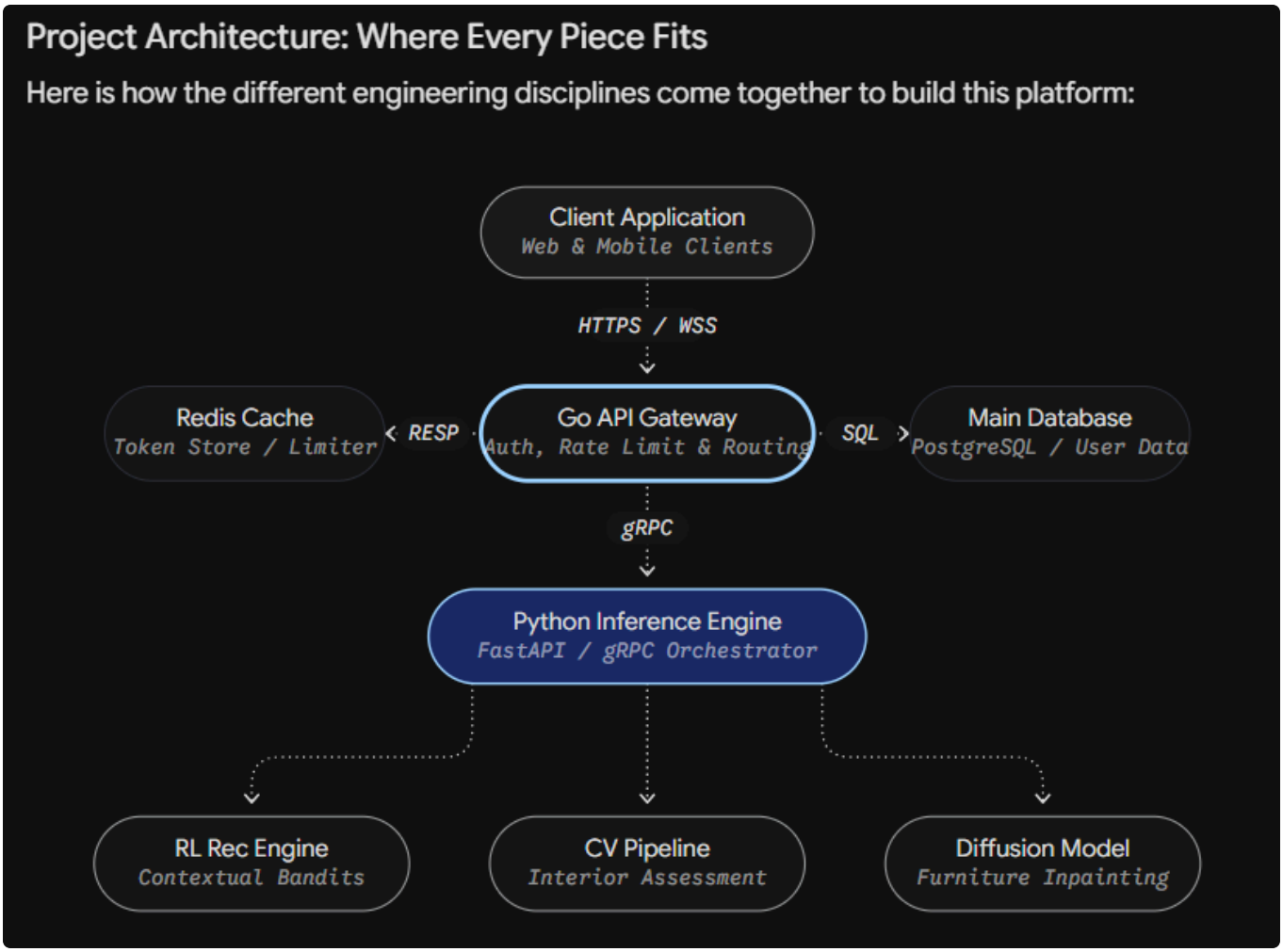
- **AI Swipe Engine:** A Tinder-style interface (swipe left/right) where the underlying machine learning model dynamically updates its weights based on your choices, continuously refining your property matches. *(Note: The initial model will be trained on synthetic data before scaling).*
- **Spatial & Furniture Prompting:** Users can input custom prompts to visualize what furniture will fit in the house, or calculate if their existing personal belongings will physically fit into the home's blueprint.
- **Deep Interior Profiling:** Provides an unprecedented look inside the house, highlighting the exact state of the interiors, existing furniture condition, and full internal blueprints.
- **Holistic Property Cards:** The main screen aggregates all critical decision-making data at a glance:
  - Price (current and historical trends)
  - Locality rating and overall crime rate
  - Commute metrics (traffic congestion, distance to subway/airport)
  - Background checks (previous owner history, legal complaints)

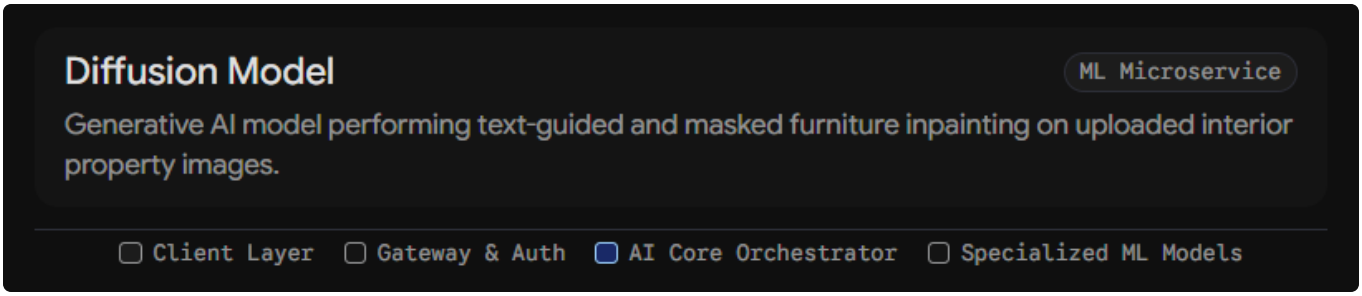
User Experience (UX) The UX transforms the tedious, data-heavy process of house hunting into a fast, highly visual, and effortless swiping experience. By spotlighting the interior layout and furniture compatibility, the app makes it easy for users to actually visualize themselves living in the space.

The real estate market is saturated with platforms offering the same basic data, leaving a massive gap for a discovery tool that actually learns user preferences and visualizes the interior reality.

Why This Idea Works (The Advantages)

- **High Engagement Loop:** Traditional property hunting is tedious and overwhelming. A gamified, Tinder-style swipe interface lowers friction, keeps users on the platform longer, and makes data collection passive and effortless.
- **Fixing the Cold Start Problem:** Starting with synthetic data allows you to train and refine the Reinforcement Learning (RL) weights immediately without waiting for a massive influx of real-world user data.
- **The "Interior Reality" Moat:** While competitors map the neighborhood, mapping the blueprints and furniture fit provides a unique value proposition that users cannot get elsewhere.





- **UI/UX Engineering:** The entry point. This handles the swipe gestures, cross-platform responsiveness (mobile and web), rendering the property cards, and providing the interface for users to type in their "furniture prompts."
- **Backend Engineering:** The brain of the operation. It acts as the API gateway, managing user authentication, handling high-volume swipe event streaming, routing requests to the AI models, and utilizing token-bucket rate limiting to prevent abuse.
- **Reinforcement Learning (RL):** The core recommendation engine. Instead of static filters, it uses algorithms (like multi-armed contextual bandits common in social media feeds) to dynamically update model weights based on whether a user swipes left or right, learning their implicit preferences over time.
- **Computer Vision (CV):** The interior analysis layer. It processes the property images to extract specific features—evaluating lighting, assessing existing furniture condition, and estimating room dimensions from standard photos.
- **Diffusion Models:** The visualization layer. This powers the "furniture prompting" feature. When a user asks "Will a modern L-shaped couch look good here?", the diffusion model in-paints the requested item directly into the empty interior photo.
- **Inference Engineering:** The critical bridge. Machine learning models are heavy. Inference engineering ensures the CV and Diffusion models run efficiently in production, handling batch processing, GPU memory management, and low-latency responses back to the Go backend.

Tech Stack Evaluation & Recommendations

Your proposed stack is highly logical for a modern, AI-heavy, high-throughput application. Here is a breakdown of why these tools fit and how to deploy them.

Domain	Recommended Stack	Implementation Notes
Backend Engineering	**Golang**	Go is the perfect choice for the API gateway. Its lightweight concurrency (`goroutines`) handles high-frequency swipe events effortlessly. Pair it with **Redis** for sub-millisecond caching and atomic rate limiting.
Inference Engineering	**Python (FastAPI / gRPC)**	You need a dedicated inference server separate from the Go backend. Use FastAPI or gRPC to serve the models. For heavy diffusion models, look into **vLLM** or **TensorRT** to optimize GPU inference speed and manage request batching.
CV, RL & Diffusion	**Python (PyTorch)**	Python is the industry standard here. Build the models in PyTorch. For the RL recommendation loop, libraries like Ray RLlib or simple contextual bandit implementations work well for dynamically updating user preference weights.
Web/UI (Multi-platform)	**Flutter**	For a swipe-heavy, gesture-driven application that needs to feel native on both mobile and web, Flutter is the strongest choice over HTML/JS/CSS or React. It provides 60fps animations out of the box, which is critical for making the Tinder-style swipe mechanics feel physically satisfying.

possible changes in tech stack we can consider :

1. Frontend: React Native vs. Current (Flutter)

While Flutter is excellent for highly customized, graphics-heavy UIs, **React Native (with Expo)** is the primary alternative for cross-platform mobile and web development.

- **Advantages of React Native:**
  - **Unified Language:** Uses JavaScript/TypeScript. If you build a web dashboard later, you can share code and use the same development team across the entire frontend.
  - **Native Components:** It renders using actual native OS components, resulting in smaller app download sizes (4-8 MB lighter than Flutter) and slightly faster cold startup times.
  - **Hiring Pool:** The market is saturated with JavaScript developers, making it significantly easier and cheaper to hire a team compared to finding specialized Dart/Flutter developers.
- **Disadvantages of React Native:**
  - **Animation Bottlenecks:** While improved with the New Architecture, it still struggles to maintain a perfect 60-120fps on complex, heavily customized animations compared to Flutter's dedicated Impeller rendering engine.
  - **UI Inconsistencies:** Because it uses native OS components, your app might look slightly different on an iPhone versus a Samsung device, requiring more platform-specific tweaking.

2. Backend Gateway: Node.js (TypeScript) vs. Current (Golang)

we will most probably work with GOLANG

Golang is a powerhouse for handling high-volume concurrent swipe events. However, **Node.js** is the most common alternative for startups building fast-iteration APIs.

- **Advantages of Node.js:**
  - **"JavaScript Everywhere" Stack:** If you switch the frontend to React Native, using Node.js allows your entire engineering team to work in a single language (TypeScript), vastly improving development velocity and code sharing.
  - **Massive Ecosystem:** The npm ecosystem is unmatched, offering pre-built packages for almost any integration (Stripe, Twilio, SendGrid, etc.) out of the box.
- **Disadvantages of Node.js:**
  - **Raw Throughput:** Node.js is single-threaded. Under extreme load (thousands of concurrent swipe events per second), it will consume more memory and CPU than Go's lightweight `goroutines`.
  - **Compute Limitations:** It is not designed for CPU-heavy tasks, meaning you must be diligent about offloading all heavy processing to your Python microservices.

3. Inference Serving: NVIDIA Triton vs. Current (FastAPI)

Using FastAPI to serve heavy machine learning models (like Diffusion for furniture and Computer Vision for interior mapping) is great for prototyping, but **NVIDIA Triton Inference Server** is the enterprise standard for production.

- **Advantages of Triton:**
  - **Dynamic Batching:** This is critical for diffusion models. If 5 users request a furniture render at the same time, Triton automatically batches them together to maximize GPU utilization and reduce wait times. You would have to code this manually in FastAPI.
  - **Multi-Model Efficiency:** Triton is designed to run multiple different models (e.g., your CV rating model and your Diffusion model) on the same GPU simultaneously without memory collisions.
  - **Higher Throughput:** It bypasses Python's Global Interpreter Lock (GIL) bottlenecks, providing significantly higher throughput and lower latency under heavy traffic.
- **Disadvantages of Triton:**
  - **Complexity:** It has a much steeper learning curve than FastAPI. It requires writing strict configuration files (`config.pbtxt`) and has fixed deployment patterns.
  - **Best Practice Compromise:** Often, the best architecture is a hybrid: keep FastAPI as a lightweight authentication/routing gateway that forwards the actual tensor computations to a Triton backend.

4. ML Framework: PyTorch (Current)

Keep this. For Computer Vision and Diffusion models, PyTorch is the undisputed industry standard. Moving to TensorFlow or JAX would severely limit your access to pre-trained models from communities like Hugging Face. The only addition should be converting your final PyTorch models into **TensorRT** or **ONNX** formats before deploying them (via Triton or FastAPI) to drastically speed up inference times.